

Maximal Independent Set (MIS) Algorithm Performance Report

MIS Project Team

May 29, 2026

1 Introduction

This report evaluates various implementations of the Maximal Independent Set (MIS) algorithm. MIS is a fundamental problem in graph theory with applications in distributed computing, wireless networks, and resource allocation. We compare a sequential implementation against parallel versions based on Luby's algorithm.

2 Algorithms

In this section, we describe the three main algorithms implemented for the Maximal Independent Set (MIS) problem.

2.1 Sequential MIS

The sequential algorithm follows a greedy approach. It iterates through all vertices and adds a vertex to the MIS if none of its neighbors have been added yet. This algorithm has a linear time complexity of $O(N + M)$, where N is the number of vertices and M is the number of edges.

```
1 for (std::size_t i = 0; i < n; i++) {  
2     if (active[i]) {  
3         mis.push_back(i);  
4         for (auto &son : g[i])  
5             active[son] = 0;  
6     }  
7 }
```

2.2 Luby's Algorithm

Luby's algorithm is a randomized parallel algorithm. It operates in rounds, where in each round, active nodes are assigned random priorities. A node is added to the MIS if it has the highest priority among its active neighbors.

```
1 while (any_active) {  
2     // 1. Assign random priorities  
3     #pragma omp parallel for  
4     for (std::size_t node = 0; node < n; node++) {  
5         if (is_active[node]) priorities[node] = dist(gen);  
6     }  
7  
8     // 2. Identify nodes with local maximum priority  
9     #pragma omp parallel  
10    {  
11        std::vector<uint32_t> local_newly_added;  
12        #pragma omp for  
13        for (std::size_t node = 0; node < n; node++) {  
14            if (!is_active[node]) continue;  
15            bool is_max = true;  
16            for (auto son : g[node]) {  
17                if (is_active[son]) {  
18                    if (priorities[son] > priorities[node] || (priorities[son] ==  
19                        priorities[node] && son > node)) {  
20                        is_max = false; break;  
21                    }  
22                }  
23            }  
24            if (is_max) local_newly_added.push_back(node);  
25        }  
26    }  
27    mis.insert(mis.end(), local_newly_added.begin(), local_newly_added.end());  
28    for (auto node : local_newly_added) is_active[node] = false;  
29 }
```

```

20     }
21 }
22 }
23     if (is_max) local_newly_added.push_back(node);
24 }
25 #pragma omp critical
26     newly_added.insert(newly_added.end(), local_newly_added.begin(),
27     local_newly_added.end());
28 }
29 // ... Update MIS and active nodes

```

2.3 Improved Luby’s Algorithm

The “Luby Improved” version optimizes the base algorithm by addressing bottlenecks in workload distribution and memory access patterns. The key improvements include:

- **Active Node Vector:** Instead of iterating over all N vertices in every round, we maintain a vector of `active_nodes`. In each round, we only process the nodes present in this vector, significantly reducing the loop overhead as more nodes become inactive.
- **Dynamic Scheduling:** For the neighbor-checking phase, we use `schedule(dynamic, 32)`. This is particularly effective for scale-free graphs where node degrees vary wildly, ensuring that threads processing high-degree nodes do not bottleneck the entire parallel region.
- **Parallel Removal:** The process of marking neighbors as inactive is parallelized using OpenMP, which was a sequential step in our base Luby implementation.
- **Efficient Filtering:** We use a parallel filter to construct the `active_nodes` vector for the next round, avoiding unnecessary work in subsequent iterations.

3 Experimental Setup

Benchmarks were performed on two primary graph types:

1. **Uniform Sparse:** Graphs with a low constant probability of edge existence.
2. **Scale-Free:** Graphs generated using the Barabási–Albert model, resembling social network structures with power-law degree distributions.

Each algorithm was tested using both standard Adjacency List (Graph) and Compressed Sparse Row (GraphCSR) formats.

4 Results

The following results were captured from the initial benchmark suite (5 graphs, 10 runs each, $N = 1,000,000$).

4.1 Standard Adjacency List Benchmarks

Algorithm	Graph Type	Mean Time	Graph Std	Avg Run Std
Sequential	Uniform Sparse	8.837 ms	0.368 ms	0.417 ms
Luby 1t	Uniform Sparse	209.911 ms	0.721 ms	3.407 ms
Luby 8t	Uniform Sparse	46.948 ms	1.792 ms	1.675 ms
Luby Improved	Uniform Sparse	27.178 ms	2.910 ms	1.929 ms
Sequential	Scale-Free	14.516 ms	0.235 ms	0.448 ms
Luby 1t	Scale-Free	113.840 ms	1.052 ms	2.173 ms
Luby 8t	Scale-Free	63.341 ms	43.632 ms	23.487 ms
Luby Improved	Scale-Free	27.630 ms	10.511 ms	10.854 ms

4.2 CSR (Compressed Sparse Row) Benchmarks

Algorithm	Graph Type	Mean Time	Graph Std	Avg Run Std
Sequential CSR	Uniform Sparse_csr	9.806 ms	1.121 ms	0.305 ms
Luby 1t CSR	Uniform Sparse_csr	192.208 ms	1.242 ms	3.607 ms
Luby 8t CSR	Uniform Sparse_csr	42.141 ms	0.312 ms	1.634 ms
Luby Improved CSR	Uniform Sparse_csr	24.641 ms	0.807 ms	2.796 ms
Sequential CSR	Scale-Free_csr	5.254 ms	0.102 ms	0.159 ms
Luby 1t CSR	Scale-Free_csr	73.781 ms	1.570 ms	2.005 ms
Luby 8t CSR	Scale-Free_csr	25.240 ms	4.114 ms	0.460 ms
Luby Improved CSR	Scale-Free_csr	14.763 ms	1.609 ms	2.322 ms

5 Conclusion

The results indicate that while Luby’s algorithm provides parallelization benefits, the Sequential implementation remains highly competitive for $N = 1,000,000$ due to lower overhead. The ”Improved” parallel version significantly outperforms the base Luby implementation by better managing workload distribution.

6 Weighted MIS

6.1 Problem Formulation

We recall that the standard Maximal Independent Set (MIS) is a subset of vertices $S \subset V$ of a graph $G = (V, E)$ such that no two vertices in S are joined by an edge, and no new vertex can be added to S without creating an edge. Now, we extend this framework by adding a weight $w_v \in \mathbb{R}^+$ for every vertex $v \in V$. In this setting, the goal is now to find a MIS with the maximum possible total weight **in practice**. It is important to note that finding the **maximum weight independent set** (i.e., the globally optimal solution) is NP-hard in general. In this view, we will focus on weight-based heuristic algorithms that find a valid MIS while favoring vertices with high weights, and then compare their solution quality empirically.

6.2 Algorithms

We decided to study the performance of the following two algorithms, and then benchmark their performance based on the initial weight assignment.

6.2.1 Algorithm 1: Weighted Greedy

A vertex v is added to the MIS if and only if it has the highest weight among its active neighbors:

$$v \text{ wins} \iff w_v \geq w_u \quad \forall u \in N(v), u \text{ active} \quad (1)$$

Ties are broken by vertex index. This approach is based on Basagni [1].

6.2.2 Algorithm 2: Weighted Sampling

Each active vertex v draws $U_v \sim \text{Uniform}(0, 1)$ and computes a priority:

$$p_v = U_v^{1/w_v} \quad (2)$$

Vertex v wins the round if $p_v > p_u$ for all active neighbors u . That is, we have that the vertices with higher weights have the higher priority.

6.3 Weight Distributions

Uniform: Each vertex v is assigned a weight drawn uniformly at random:

$$w_v \sim \text{Uniform}(1, 10) \quad (3)$$

Exponential: Each vertex v is assigned:

$$w_v = 1 + X, \quad X \sim \text{Exponential}(\lambda) \quad (4)$$

The $+1$ shift ensures $w_v > 0$, which is required by the power trick in Algorithm 2.

Clustered: A fraction $\rho = 0.1$ of vertices are randomly designated as hotspots:

$$w_v = \begin{cases} 10 & \text{with probability } \rho \\ 1 & \text{otherwise} \end{cases} \quad (5)$$

6.4 Experimental Results

The following results were captured from the weighted MIS benchmark suite ($N = 1,000,000$, 5 graphs, 10 runs each).

Algorithm	Graph Type	Mean Time	Graph Std	Avg Run Std	Mean Weight
Weighted Greedy	Uniform Sparse (uniform)	48.346 ms	10.319 ms	19.023 ms	221,205.58
Weighted Sampling	Uniform Sparse (uniform)	101.725 ms	62.736 ms	86.907 ms	169,862.79
Weighted Greedy	Scale-Free (uniform)	34.835 ms	5.984 ms	7.885 ms	2,220,850.60
Weighted Sampling	Scale-Free (uniform)	65.297 ms	47.481 ms	41.514 ms	1,998,436.76
Weighted Greedy	Uniform Sparse (exp)	44.856 ms	2.182 ms	7.825 ms	183,315.81
Weighted Sampling	Uniform Sparse (exp)	65.743 ms	33.578 ms	24.710 ms	102,926.19
Weighted Greedy	Scale-Free (exp)	53.372 ms	44.933 ms	35.131 ms	1,338,124.13
Weighted Sampling	Scale-Free (exp)	40.659 ms	5.625 ms	11.284 ms	1,133,123.09
Weighted Greedy	Uniform Sparse (clustered)	63.872 ms	4.375 ms	7.339 ms	163,565.00
Weighted Sampling	Uniform Sparse (clustered)	45.382 ms	3.298 ms	5.214 ms	107,985.08
Weighted Greedy	Scale-Free (clustered)	41.219 ms	12.018 ms	25.399 ms	1,051,920.60
Weighted Sampling	Scale-Free (clustered)	62.458 ms	25.564 ms	44.257 ms	877,655.86

6.5 Discussion

The results indicate that the Greedy approach consistently finds the MIS with higher total weight across all graph types and initial weight distributions. This can be explained by the fact that stochastic sampling can occasionally make the algorithm give a low weight neighboring vertex a higher priority, thus permanently excluding the high weight vertex from the MIS selection.

We also see that on Scale-Free graph types, the mean weight results are much closer than that of on the Uniform-Sparse graph types (gap of 11–20% on Scale-Free vs. 30–78% on Uniform Sparse).

We see that the exponential and clustered initial weight distributions improve greedy’s advantage (78% and 52% gaps respectively on Uniform Sparse, vs. 30% for uniform weights). The reason for this is that in these settings, we have relatively less very high-weight vertices, and the greedy approach always captures them. Meanwhile, the sampling approach may exclude them in case a low-weight vertex is selected instead at random.

References

- [1] S. Basagni. Finding a maximal weighted independent set in wireless networks. Telecommunication Systems, 18(1–3):155–168, 2001. URL <http://dx.doi.org/10.1023/A:1016747704458>.